Module: ACID Tables & Hive UDFs (using customers & transactions tables)

Part 1: ACID & Transactional Tables

Concept Overview

Hive supports **ACID (Atomicity, Consistency, Isolation, Durability)** properties for **transactional tables** starting from Hive 0.14+.
They allow:

- INSERT, UPDATE, DELETE operations (like RDBMS)
- Small transactions with row-level locking

Pre-requisites

Make sure these are set before creating ACID tables:

```
SET hive.cli.print.current.db=true;

SET hive.cli.print.header=true;
```

```
SET hive.support.concurrency = true;
SET hive.txn.manager = org.apache.hadoop.hive.ql.lockmgr.DbTxnManager;
SET hive.enforce.bucketing = true;
```

Why Bucketing is Required in Transactional Tables

- **Hive ACID tables** support INSERT, UPDATE, DELETE — so Hive must **track each row** precisely.
- **Bucketing** ensures each row always goes into the **same file (bucket)** based on a hash of the key (e.g., cust_id).

- This makes **updates and deletes faster**, since Hive can find and modify only the affected bucket, not the whole table.

- **ORC + Bucketing** are **mandatory** for transactional tables because ORC stores row-level metadata and bucketing gives deterministic storage.

Delta Files in Hive ACID Tables

- When you **UPDATE** or **DELETE**, Hive doesn't rewrite the whole ORC file.

- It creates **delta files** (like delta_0002_0003) for each bucket to store only the changes.

- The **base** folder has the original data, and Hive **merges base + delta** files when you query — showing the latest data.

- Over time, many deltas are **compacted** into a new base file for better performance.

Create an ACID (Transactional) Table

We'll convert the existing customers table into a **transactional ORC table**.

```
CREATE TABLE customers_txn (
   cust_id INT,
   fname STRING,
   lname STRING,
   age INT,
   profession STRING
)
CLUSTERED BY (cust_id) INTO 4 BUCKETS
STORED AS ORC
TBLPROPERTIES ('transactional'='true');
```

Load data:

```
INSERT INTO customers_txn SELECT * FROM customers;
```

Demonstrate Update & Delete

- Update Example

```
UPDATE customers
SET profession = 'Senior Engineer'
WHERE cust_id = 104;

UPDATE customers_txn
SET profession = 'Senior Engineer'
WHERE cust_id =4000005;
```

- Delete Example

```
DELETE FROM customers_txn
WHERE age < 25;
```

- Insert Example

```
INSERT INTO customers_txn VALUES ( 4000005, 'Karen', 'Kale', 28, 'Analyst');
```

Check Results

```
SELECT * FROM customers_txn;
```

Compacting Transactions

Compaction merges delta files after updates/deletes for efficiency.

```
SET hive.compactor.initiator.on = true;

SET hive.compactor.worker.threads = 1;


ALTER TABLE customers_txn COMPACT 'major';
SHOW COMPACTIONS;
```

Part 2: Hive UDFs (User Defined Functions)

Concept Overview

Hive UDFs allow you to extend Hive by writing custom logic in Java or Python (using Hive's TRANSFORM clause).

There are 3 types:

1. **UDF (Scalar)** → Works on single row
2. **UDAF (Aggregate)** → Works on multiple rows
3. **UDTF (Table Generating)** → Returns multiple rows for each input row

---

Built-in UDF Examples

Use your transaction table for practice:

```
-- Uppercase conversion
        SELECT UPPER(fname), LOWER(lname) FROM customers;

-- String length
SELECT fname, LENGTH(fname) AS name_length FROM customers;

-- Date functions
SELECT txn_date, YEAR(txn_date), MONTH(txn_date) FROM transactions;

-- Conditional logic
SELECT fname, IF(age > 30, 'Senior', 'Junior') AS category FROM customers;
```

Custom UDF (using TRANSFORM & Python)

Convert all customer first names to **uppercase** using a Python script.

Step 1: Python Script

Save this as /home/cdacoctnov012/scripts/upper_name.py

```
#!/usr/bin/env python3
import sys
for line in sys.stdin:
    fname = line.strip()
    print(fname.upper())
```

Step 2: Hive Query

```
ADD FILE /home/cdacoctnov012/scripts/upper_name.py;

SELECT TRANSFORM (fname)
USING 'python3 upper_name.py'
AS (upper_fname)
FROM customers;
```

Suppose you want to **mask customer names** using Python.

```
ADD FILE /home/cdacoctnov012/scripts/mask_name.py;

SELECT TRANSFORM (fname, lname)
USING 'python3 mask_name.py'
AS (masked_fname, masked_lname)
FROM customers;
```

mask_name.py

```
#!/usr/bin/env python3
import sys
for line in sys.stdin:
    parts = line.strip().split('\t')
    if len(parts) == 2:
        fname, lname = parts
        print(fname[0] + "***", lname[0] + "***")
```